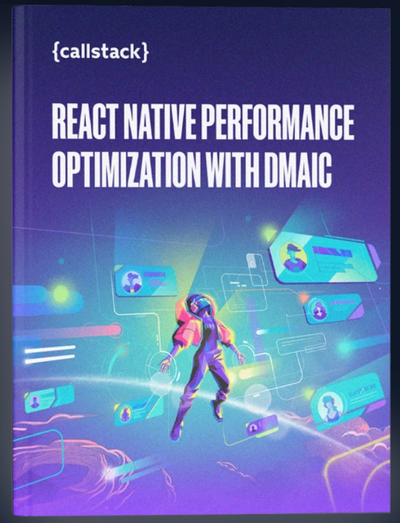
3 - ZÁKLADY REACT NATIVE

Lukáš Mikolaj



REACT NATIVE OPTIMIZATION



História

ENIAC

Prvý programovateľný elektronický počítač

1945 · miestnosť plná hardvéru · programovanie káblami a prepínačmi

OTÁZKA

Čo bolo podľa vás sloganom/mottom Javy od počiatku?

1. Strong typing, strong performance
2. From mainframes to microservices
3. Write once, run anywhere

OTÁZKA

Približne o ďalších 20 rokov sa stal React Native open source projektom a jeho mottom bolo:

1. Write once, run anywhere
2. Learn once, write anywhere
3. One framework, all platforms

Kde sa dá spustiť React Native?











Smart TV

VR

Typy mobilných aplikácií

Natívne

Vyvíjané priamo pre konkrétnu platformu.

Webové

Bežia v prehliadači a nasadzujú sa ako web.

Hybridné

Web UI zabalené v natívnom kontajneri.

Natívne aplikácie

Čo to znamená

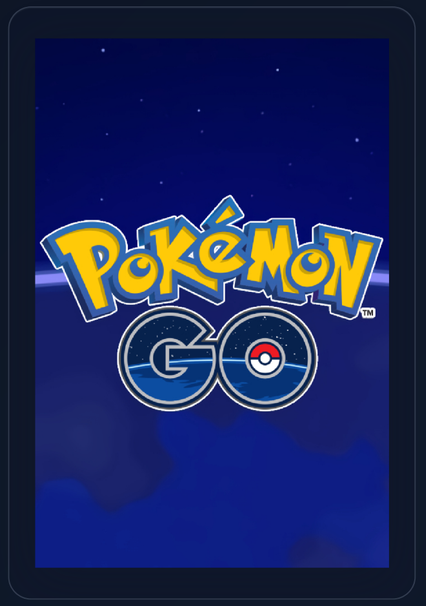
- Swift v Xcode pre iOS
- Kotlin v Android Studio pre Android
- Priamy prístup k natívnym API

Vlastnosti

- Najvyšší výkon a stabilita
- Veľmi dobrý používateľský zážitok
- Samostatný vývoj pre každú platformu

Príklad: Natívne aplikácie







Hra, ktorá potrebuje výkon, senzory, kameru, GPS a priamu integráciu so zariadením.

Webové aplikácie

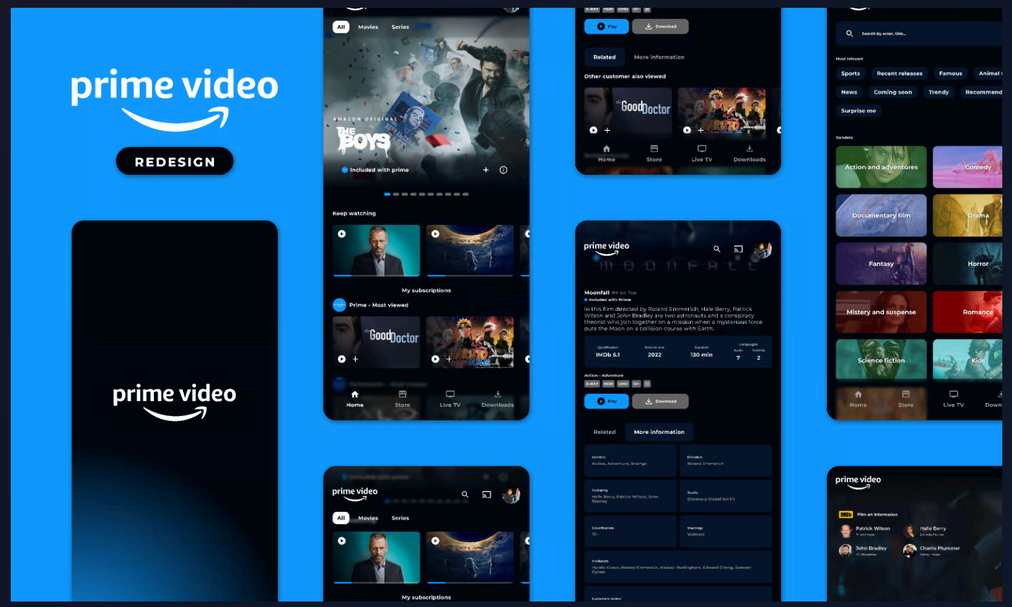
Čo to znamená

- HTML, CSS a JavaScript
- Spúšťanie cez URL alebo QR kód
- Aktualizácie nasadením na server

Vlastnosti

- Jednoduchá distribúcia
- Prenositeľnosť medzi platformami
- Limity browseru, pripojenia a natívnych API

Príklad: Hybridné aplikácie



Obsah a rozhranie sa vedia rýchlo meniť naprieč platformami, no stále záleží na možnostiach runtime.

Hybridné aplikácie

Čo to znamená

- Web UI v natívnom kontajneri
- Distribúcia cez app store
- Wrapper na prístup k hardvéru a natívnym API

Vlastnosti

- Jeden kód pre viac platforiem
- Rýchlejší vývoj a údržba
- WebView overhead pri náročnejšom UI

Príklad: Hybridné aplikácie



Aplikácia, kde je dôležitá rýchla iterácia UI a zdieľanie časti aplikačnej logiky.

OTÁZKA

Do ktorej z týchto kategórií patrí aplikácia vyvíjaná v React Native?

1. Natívne aplikácie
2. Webové aplikácie
3. Hybridné aplikácie

Greenfield vs Brownfield aplikacie

Opisat ako je mozne integrovat RN do existujucej aplikacie a pridaj obrazok z wordu https://www.callstack.com/blog/brownfield-development-with-react-native

Podkategórie mobilných aplikácií

- **Natívne aplikácie** → Cross-platform aplikácie
- **Webové aplikácie** → Progresívne webové aplikácie (PWA)
- **Hybridné aplikácie**

React Native patrí medzi cross-platform technológie pre natívne aplikácie.

Aplikácie používajúce Expo

Zoznam: evanbacon.dev/blog/expo-apps

React Native Under the hood

Old Architecture



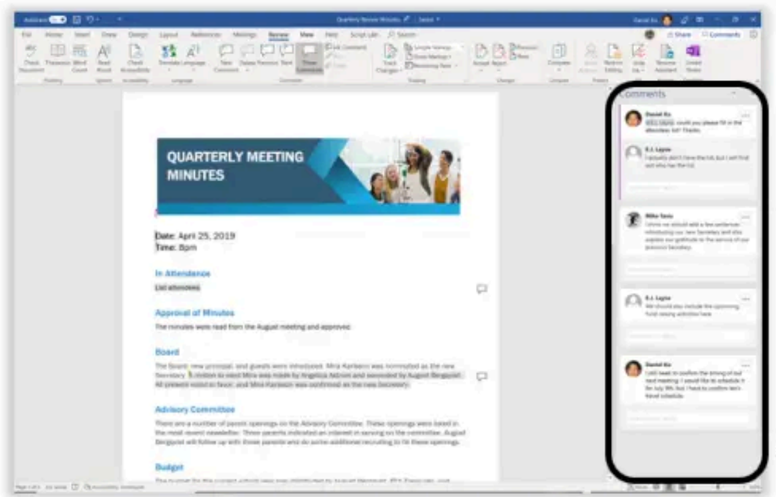
New Architecture



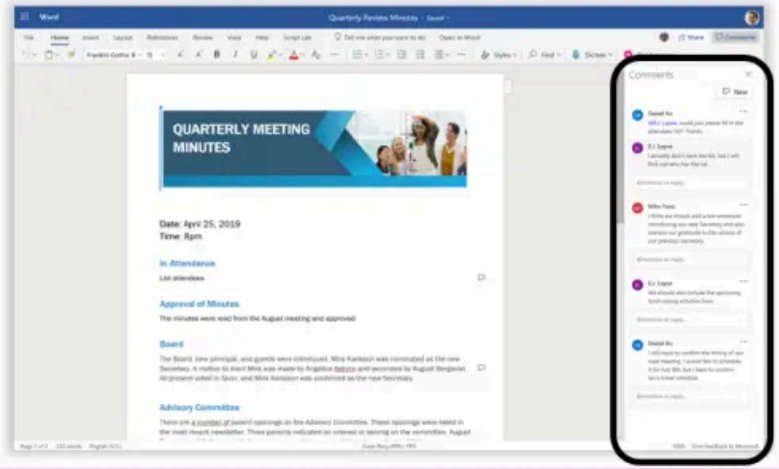
JSI as an Interface



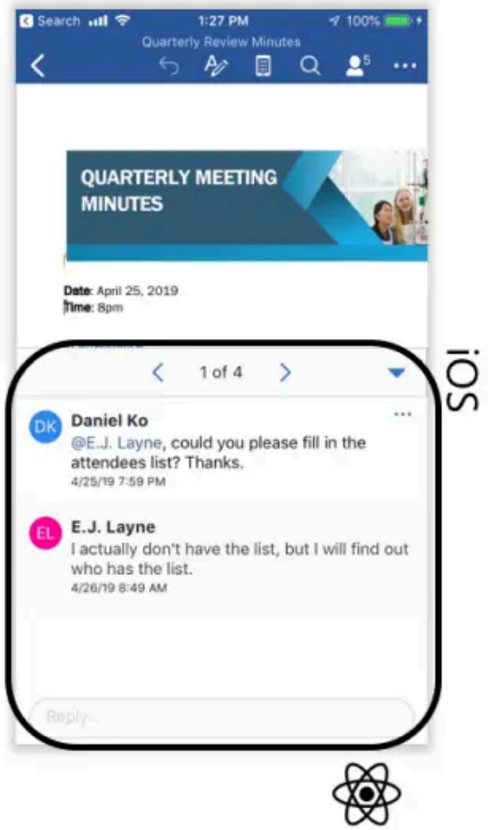
Microsoft uses React and React Native



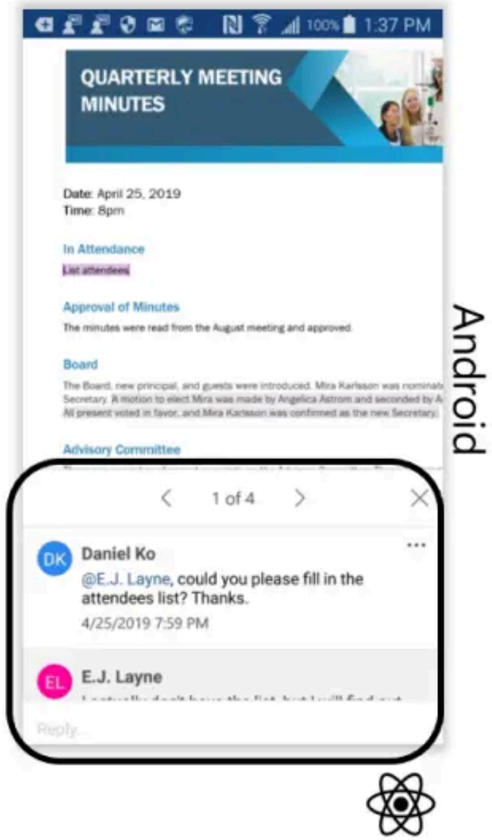
Windows



Web



iOS



Android

View Flattening

React skladá strom elementov. Uzly, ktoré ovplyvňujú len layout a nič vizuálne nevykreslia, sú **layout-only**.

```
function MyComponent() {
  return (
    <View>
      <View style={{ margin: 10 }}>
        <View style={{ padding: 10 }}>
          <Image {...props} />
          <Text>This is a title</Text>
        </View>
      </View>
    </View>
  );
}
```

BEST PRACTICE

View Flattening: príklad

1. Pôvodný strom

Viacero `View` komponentov slúži iba na layout.

2. Extrakcia komponentu

`MyInnerComponent` obalí vnútorný layout, ale wrappery stále ostávajú.

3. Manuálny View Flattening

Štýl posunieme ako prop a odstránime zbytočný vonkajší `View`.

STAVEBNÉ BLOKY UI

ZÁKLADNÉ KOMPONENTY V REACT NATIVE

Základné komponenty React Native

- ActivityIndicator
- Button
- FlatList
- Image
- KeyboardAvoidingView
- Modal
- Pressable
- RefreshControl
- ScrollView
- SectionList
- StatusBar
- StyleSheet
- Switch
- Text
- TextInput
- TouchableHighlight
- TouchableOpacity
- TouchableWithoutFeedback
- View
- VirtualizedList

VIEW

Základný stavebný blok UI

`View` organizuje a rozkladá ostatné komponenty. Je to najbližší ekvivalent webového `<div>`, ale na každej platforme sa zmení na natívny prvok.

iOS	Android	Web
UIView	**android.View**	** **

```
import { View } from 'react-native';

<View style={styles.container}>
  {/* ďalšie komponenty */}
</View>
```

VIEW

Vybrané props

`onLayout`

Spustí sa pri mounte a pri zmene rozmerov komponentu.

```
import { View } from 'react-native';

<View
  onLayout={(event) => {
    const { x, y, width, height } =
      event.nativeEvent.layout;

    console.log({ x, y, width, height });
  }}
/>
```

`accessible` + `accessibility*`

Pomáhajú asistenčným technológiám pochopiť význam a obsah prvku.

```
<View
  accessible
  accessibilityLabel="Hlavný navigačný panel"
>
  {/* obsah */}
</View>
```

StyleSheet

Statické properties

- `absoluteFill` vyplní celý screen
- `hairlineWidth` vytvorí najtenšiu viditeľnú čiaru

Metódy

- `create()` typovo skontroluje štýly
- `flatten()` zloží pole štýlov
- `compose()` skombinuje štýly

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 16,
    backgroundColor: '#ffffff',
  },
});
```

BEST PRACTICE

Kedy `StyleSheet.flatten()` nepotrebujeme?

Zbytočné

```
const textStyle = StyleSheet.flatten([
  styles.text,
  type === 'link' ? styles.link : null,
]);

return <Text style={textStyle} />;
```

Základné komponenty pole štýlov spracujú samy.

Jednoduchšie

```
return (
  <Text
    style={[
      styles.text,
      type === 'link' && styles.link,
    ]}
  />
);
```

Správne použitie

```
return (
  <Carousel
    style={styles}
    width={styles.width}
  />
);
```

Flatten použi vtedy, keď potrebuješ hodnotu zo štýlu prečítať.

OTÁZKA

Čo vyrenderuje React na webe?

```
<div>
  It's easy with Localhost!
</div>
```

1.

Text „It's easy with Localhost!"

2.

Nič, pretože `div` nemá `children`

3.

Nič, pretože nastane error

Správne: 1. *Text je na webe platným child elementom* `div`.

OTÁZKA

A čo rovnaký obsah v React Native?

```
import { View } from 'react-native';

<View>
  It's easy with Localhost!
</View>
```

1.	2.	3.
Zobrazí sa text	Nevyrenderuje sa nič	Aplikácia zobrazí error

```
Text strings must be rendered within a <Text> component.
```

Každý textový obsah musí byť vo vnútri `<Text>`.

TEXT

Text: interakcia, riadky a štýly

Reaguje na stlačenie

`onPress` spraví text interaktívnym.

```
import { Text } from 'react-native';

<Text
  onPress={() => openDetails()}
  style={{ color: 'blue' }}
>
  Otvor detail
</Text>
```

Obmedzí počet riadkov

`numberOfLines` skráti dlhý text.

```
import { Text } from 'react-native';

<Text numberOfLines={1}>
    Tento dlhý text sa skrátí a bude mať tri bod
</Text>
```

Vnorený Text dedí štýly

Vnorený text preberá štýl rodiča a môže ho prepísať.

```
import { Text } from 'react-native';

<Text style={{ fontFamily: 'Calibri', fontSize:
  Základný text
  <Text style={{ fontSize: 20, fontWeight: 'bold
    Zvýraznený nadpis
  </Text>
  Pokračovanie základného textu
</Text>
```

TEXT

Dva kontajnery, dva layouty

Text layout

```
<Text>
  <Text>Prvá časť a </Text>
  <Text>druhá časť</Text>
</Text>
```

```
| Prvá časť a druhá |
| časť |
```

Obsah tečie ako jeden text a vnorené texty dedia štýly.

Flexbox layout

```
<View>
  <Text>Prvá časť a </Text>
  <Text>druhá časť</Text>
</View>
```

```
| Prvá časť a |
| druhá časť |
```

Každý `Text` je samostatný obdĺžnik vo flexboxe.

BEST PRACTICE

Čo je zlé na tejto podmienke?

```
<View>
  {items.length && <ListHeader />}
</View>
```

Keď je pole prázdne

`items.length` je `0` a výraz `&&` vráti práve túto hodnotu.

`0` mimo komponentu `<Text>`

Bezpečné alternatívy

```
items.length > 0 && <ListHeader />

Boolean(items.length) && <ListHeader />

items.length ? <ListHeader /> : null
```

IMAGE

Lokálny alebo vzdialený zdroj

`source`

Podporuje lokálny asset ale aj `uri` internetového zdroja.

```
import { Image } from 'react-native';

<Image
  source={{ uri: product.image }}
  style={styles.image}
/>

<Image
  source={require('./assets/logo.png')}
  style={styles.image}
/>
```

`src` a `srcSet`

Internetové zdroje podobné webovému API. Ak zadáme aj `source`, `src` má prioritu.

```
import { Image } from 'react-native';

<Image
  src="https://example.com/logo.png"
  srcSet="logo.png 1x, logo@2x.png 2x"
/>
```

Pri internetovom obrázku vždy určte rozmery.

PRESSABLE

Životný cyklus dotyku

Krátky dotyk

`onPressIn` dá vizuálny feedback, po pustení nasleduje `onPressOut` a hlavná akcia `onPress`.

`onPressIn` → `onPressOut` → `onPress`

Dlhý dotyk

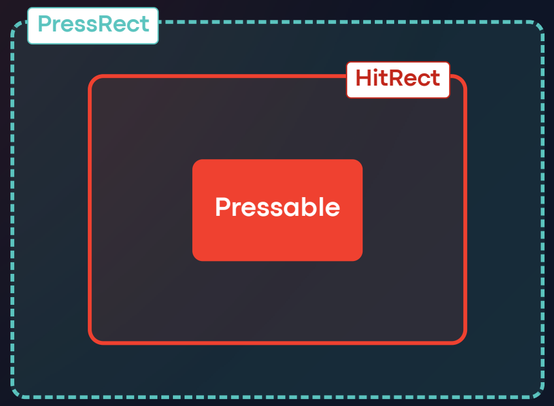
Po uplynutí `delayLongPress` sa zavolá `onLongPress`, potom `onPressOut`.

Predvolený delay je 500 ms.

`onPressIn` → `onLongPress` → `onPressOut`

PRESSABLE

Malé tlačidlo, väčšia dotyková plocha



`hitSlop`

Zväčší plochu, na ktorej sa dotyk začne, bez zmeny vizuálnej veľkosti.

`pressRetentionOffset`

Určí, ako ďaleko môže prst odísť a dotyk stále ostane aktívny.

```
<Pressable
  hitSlop={10}
  pressRetentionOffset={20}
  onPress={handlePress}
/>
```

PRESSABLE

Stav dotyku patrí do vizuálneho feedbacku

```
<Pressable
  disabled={isSaving}
  onPress={save}
  style={({ pressed }) => [
    styles.button,
    pressed && styles.buttonPressed,
    isSaving && styles.buttonDisabled,
  ]}
>
  <Text>Uložiť</Text>
</Pressable>
```

Funkcia ako `style`

Dostane stav `pressed` a môže okamžite zmeniť vzhľad.

`disabled`

Zablokuje všetky dotykové interakcie komponentu.

TEXTINPUT

Controlled input

1. Hodnota je v stave

`value` určuje, čo input zobrazuje.

2. Zmena aktualizuje stav

`onChangeText` dostane priamo nový text.

```
const [name, setName] = useState('');

return (
  <View>
    <TextInput
      value={name}
      onChangeText={setName}
      placeholder="Zadaj meno"
    />

    <Text>Aktuálna hodnota: {name}</Text>
  </View>
);
```

TEXTINPUT

Klávesnica, focus a metódy

Správanie

- `secureTextEntry`
- `autoCapitalize`
- `autoCorrect`

Klávesnica

- `email-address`
- `phone-pad`
- `numeric`
- `url`

Imperatívne metódy

- `focus()`
- `blur()`
- `clear()`
- `isFocused()`

```
<TextInput
  keyboardType="email-address"
  autoCapitalize="none"
  onFocus={() => setFocused(true)}
  onBlur={() => setFocused(false)}
/>
```

CONTEXT

Props drilling: stav putuje cez celý strom

Každá vrstva odovzdáva props ďalej

App — vlastní theme

theme, setTheme

Page — iba posúva props

theme, setTheme

Toolbar — iba posúva props

theme, setTheme

ThemeButton — používa stav

Page a Toolbar hodnotu nepotrebujú. Len ju posúvajú nižšie.

```
import { useState } from 'react';
  import { Pressable, Text } from 'react-native';

  function App() {
    const [theme, setTheme] = useState('light');
    return <Page theme={theme} setTheme={setTheme} />;
  }

function Page({ theme, setTheme }) {
  return <Toolbar theme={theme} setTheme={setTheme} />;
}

function Toolbar({ theme, setTheme }) {
  return <ThemeButton theme={theme} setTheme={setTheme} />;
}

function ThemeButton({ theme, setTheme }) {
  return (
    <Pressable onPress={() => setTheme(theme === 'light' ? 'dark' : 'light')}>
      <Text>Téma: {theme}</Text>
    </Pressable>
  );
}
```

CONTEXT

Context: consumer číta hodnotu priamo

Provider sprístupní hodnotu podstromu

App — theme + setTheme

ThemeContext.Provider

Page

bez theme props

Toolbar

bez theme props

ThemeButton — useContext(ThemeContext)

Context odstráni posúvanie props cez komponenty, ktoré hodnotu nepoužívajú.

```
import { createContext, useContext, useState } from 'react';
  import { Pressable, Text } from 'react-native';

  const ThemeContext = createContext(null);

function App() {
  const [theme, setTheme] = useState('light');
  const value = { theme, setTheme };

  return (
    <ThemeContext.Provider value={value}>
      <Page />
    </ThemeContext.Provider>
  );
}

function Page() {
  return <Toolbar />;
}

function Toolbar() {
  return <ThemeButton />;
}

function ThemeButton() {
  const { theme, setTheme } = useContext(ThemeContext);
  return (
    <Pressable onPress={() => setTheme(theme === 'light' ? 'dark' : 'light')}>
      <Text>Téma: {theme}</Text>
    </Pressable>
  );
}
```

Keď sa zmení hodnota Contextu, jeho consumeri sa znovu vykreslia.

CONTEXT · BEST PRACTICE

Kedy použiť Context?

Záleží podľa toho, kto hodnotu potrebuje a ako často sa mení.

Dobrá voľba

Zdieľaná hodnota, ktorú potrebuje viacero vzdialených častí UI.

- Téma
- Jazyk
- Prihlásený používateľ

Pozor na zmeny

Keď sa hodnota Contextu zmení, znovu sa vykreslia jeho consumeri.

Často meniaci sa stav môže spúšťať zbytočné rendery.

Oddeľ nezávislý stav

Nespájaj spolu hodnoty, ktoré potrebujú odlišné komponenty.

`filter` a `todos` môžu mať samostatné Contexty alebo store so selectormi.

```
import { useState } from 'react';
import { View } from 'react-native';

const App = () => {
  const [filter, setFilter] = useState('all');
  const [todos, setTodos] = useState([]);
  const filteredTodos = todos.filter((todo) => {
    if (filter === 'active') return !todo.completed;
    if (filter === 'completed') return todo.completed;
    return true;
  });

  return (
    <View>
      {['all', 'active', 'completed'].map((filterType) => (
        <FilterMenuItem
          key={filterType}
          title={filterType}
          currentFilter={filter}
          onChange={setFilter}
        />
      ))}
      {filteredTodos.map((todo) => (
        <TodoItem
          key={todo.id}
          item={todo}
          onChange={setTodos}
        />
      ))}
    </View>
  );
};
```

CONTEXT · BEST PRACTICE

Stabilizuj referenciu v Contexte

Nový objekt pri každom renderi znamená novú hodnotu Contextu, aj keď sa jeho "polia" nezmenili.

Nový objekt pri každom renderi

```
function ThemeProvider({ children }) {
  const [isDarkMode, setIsDarkMode] = useState(false);

  const value = {
    isDarkMode,
    toggleTheme: () => setIsDarkMode(mode => !mode),
  };

  return (
    <ThemeContext.Provider value={value}>
      {children}
    </ThemeContext.Provider>
  );
}
```

Pri každom renderi vznikne nová referencia `value`.

Rovnaká hodnota, stabilná referencia

```
import { useCallback, useMemo, useState } from 'react';

function ThemeProvider({ children }) {
  const [isDarkMode, setIsDarkMode] = useState(false);

  const toggleTheme = useCallback(() => {
    setIsDarkMode(mode => !mode);
  }, []);

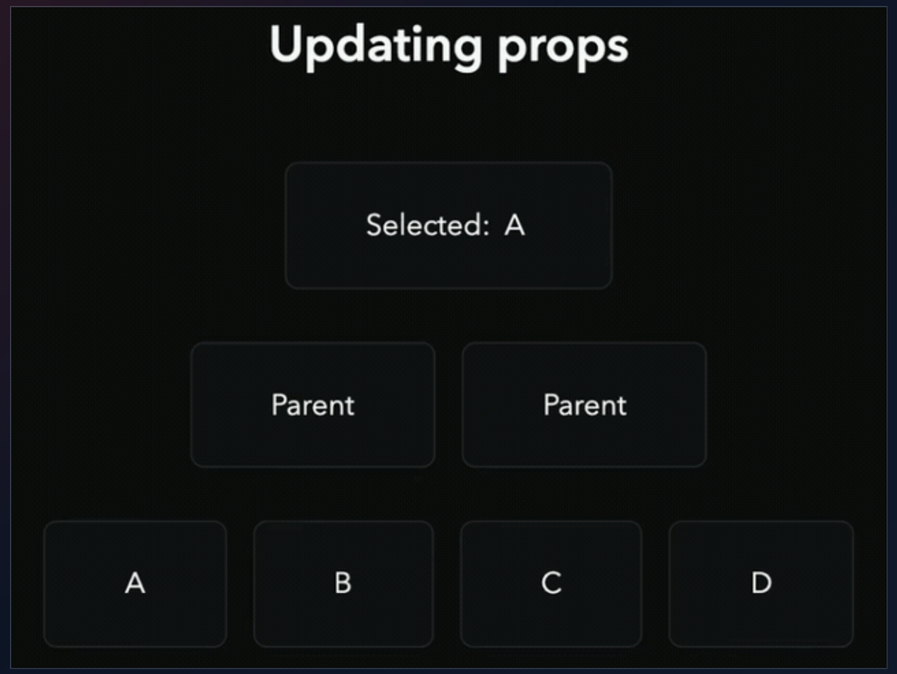
  const value = useMemo(
    () => ({ isDarkMode, toggleTheme }),
    [isDarkMode, toggleTheme]
  );

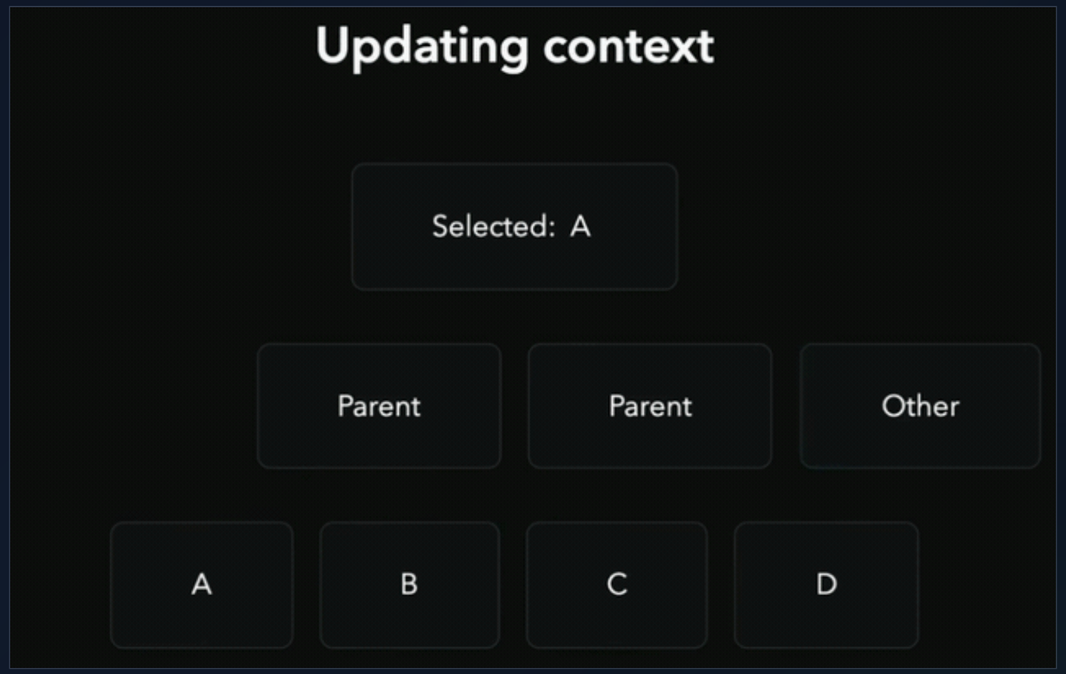
  return (
    <ThemeContext.Provider value={value}>
      {children}
    </ThemeContext.Provider>
  );
}
```

Referencia `value` zostane stabilná, kým sa nezmení `isDarkMode`.

REACT STATE

Context vs props drilling





APP DEMO

SCROLLVIEW

Ohranič priestor, scrolluj obsah

Má dve dôležité vlastnosti

`ScrollView` zobrazí obsah vo vlastnom posúvateľnom okne.

- Výška musí byť ohraničená cez rodičovský layout, často pomocou `flex: 1`.
- Všetky deti vytvorí naraz, preto ho používaj pre menší alebo obmedzený obsah.

Pre dlhé zoznamy použi `FlatList` alebo `SectionList`.

```
import { ScrollView, Text, View } from 'react-native';

function ArticleScreen() {
  return (
    <View style={{ flex: 1 }}>
      <ScrollView
        style={{ flex: 1 }}
        contentContainerStyle={{ padding: 16 }}
      >
        <Text>Úvod článku</Text>
        <Text>Ďalší obsah článku</Text>
      </ScrollView>
    </View>
  );
}
```

SCROLLVIEW

Nastav smer a obsah

Smer

`horizontal` prepne zoznam na vodorovné posúvanie.

Indikátor

`showsHorizontalScrollIndicator` a `showsVerticalScrollIndicator` ovládajú zobrazenie indikátora.

Vnútorný kontajner

`contentContainerStyle` upravuje obal obsahu, nie samotný viewport.

```tsx
import { ScrollView, Text } from 'react-native';

const categories = ['Novinky', 'Šport', 'Kultúra'];

function CategoryRow() {
  return (
    <ScrollView
      horizontal
      style={{ height: 64 }}
      showsHorizontalScrollIndicator={false}
      contentContainerStyle={{
        flexDirection: 'row',
        gap: 12,
        paddingHorizontal: 16,
      }}
    >
      {categories.map((category) => (
        <Text key={category} style={{ padding: 12 }}>
          {category}
        </Text>
      ))}
    </ScrollView>
  );
}
```

SCROLLVIEW

Stránkovanie a scroll eventy

Stránkuj obsah

`pagingEnabled` posúva obsah po stránkach. Pri vodorovnom scrollovaní každá strana má šírku viewportu.

Sleduj pozíciu

`onScroll` dostáva aktuálny offset; `scrollEventThrottle` určuje frekvenciu udalostí.

Scroll eventy môžu prichádzať každý frame. Vyhni sa zbytočnej práci v handlery.

```
import { useRef } from 'react';
import { Animated, Text, View, useWindowDimensions } from 'react-
native';

function PagedIntro() {
  const { width } = useWindowDimensions();
  const scrollX = useRef(new Animated.Value(0)).current;

  return (
    <View style={{ flex: 1 }}>
      <Animated.ScrollView
        horizontal
        style={{ flex: 1 }}
        pagingEnabled
        scrollEventThrottle={16}
        onScroll={Animated.event(
          [{ nativeEvent: { contentOffset: { x: scrollX } } }],
          { useNativeDriver: true }
        )}
      >
        <View style={{ width, height: 320 }}>
          <Text>Strana 1</Text>
        </View>
        <View style={{ width, height: 320 }}>
          <Text>Strana 2</Text>
        </View>
      </Animated.ScrollView>
    </View>
  );
}
```

SCROLLVIEW — PROBLÉM PRI VEĽKÝCH DÁTACH

FLATLIST

Vyrenderuj len viditeľné položky

Virtualizovaný zoznam

`FlatList` postupne vytvára položky v okolí viditeľnej oblasti - viewportu.

- Vhodný pre dlhé alebo dynamické zoznamy.
- `data` obsahuje položky a `renderItem` opisuje jednu položku.
- Na rozdiel od `ScrollView` nevytvára celý obsah naraz.

```tsx
import { FlatList, Text, View } from 'react-native';

const people = [
  { id: '1', name: 'Nina' },
  { id: '2', name: 'Adam' },
  { id: '3', name: 'Lea' },
];

function PeopleScreen() {
  return (
    <FlatList
      data={people}
      keyExtractor={(person) => person.id}
      renderItem={({ item }) => (
        <View>
          <Text>{item.name}</Text>
        </View>
      )}
    />
  );
}
```

FLATLIST

Prispôsob obsah zoznamu

Časti zoznamu

- `ListHeaderComponent` pridá obsah pred položky.
- `ListEmptyComponent` sa zobrazí, keď je pole prázdne.
- `ListFooterComponent` pridá obsah za položky.
- `numColumns` zmení zoznam na mriežku.

```
<FlatList
  data={tasks}
  numColumns={2}
  columnWrapperStyle={{ gap: 12 }}
  keyExtractor={(task) => task.id}
  renderItem={({ item }) => (
    <TaskRow task={item} />
  )}
  ListHeaderComponent={<Text>Moje úlohy</Text>}
  ListEmptyComponent={<Text>Zatiaľ nič</Text>}
  ListFooterComponent={<Text>Koniec</Text>}
/>
```

FLATLIST

Refresh a načítaj ďalšie dáta

Pull to refresh

`refreshing` riadi indikátor a `onRefresh` spustí obnovenie.

Načítanie ďalšej stránky

`onEndReached` sa zavolá pri priblížení ku koncu. `onEndReachedThreshold` určuje, ako skoro.

```
<FlatList
  data={items}
  renderItem={({ item }) => (
    <Text>{item.name}</Text>
  )}
  keyExtractor={(item) => item.id}
  refreshing={isRefreshing}
  onRefresh={refreshItems}
  onEndReached={loadNextPage}
  onEndReachedThreshold={0.5}
  ListFooterComponent={
    isLoadingMore
      ? <ActivityIndicator />
      : null
  }
/>
```

FLATLIST

Ref metódy

- `scrollToIndex()` posunie list na položku.
- `scrollToOffset()` posunie list na konkrétny offset.
- `initialNumToRender` určuje počet položiek pri prvom vykreslení.
- `windowSize` nastavuje veľkosť vykresľovaného okolia.

Výšku okna meň až po meraní. Menšie okno šetrí zdroje, no pri rýchlom scrollovaní môže ukázať prázdne miesto.

```tsx
const listRef = useRef<FlatList<Task>>(null);

function TaskList() {
  return (
    <View>
      <Button
        title="Ukaž úlohu 21"
        onPress={() => listRef.current?.scrollToIndex({
          index: 20,
          animated: true,
        })}
      />
      <FlatList
        ref={listRef}
        data={tasks}
        renderItem={({ item }) => <TaskRow task={item} />}
        keyExtractor={(task) => task.id}
        initialNumToRender={8}
        windowSize={7}
      />
    </View>
  );
}
```

FLATLIST · BEST PRACTICE

Keď sa zmení referencia callbacku

Čo sa deje pri renderi?

- Inline funkcia `renderItem` vznikne pri každom renderi rodiča nanovo.
- `FlatList` porovnáva props plytko-shalow equal; nová referencia je zmena - render.
- List môže znovu spracovať viditeľné položky, takzvaná recyklácia.

```
function TaskScreen({ tasks }) {
  const [showHint, setShowHint] = useState(false);

  return (
    <View>
      <Button
        title="Tip"
        onPress={() => setShowHint((value) => !value)}
      />
      {showHint && <Text>Tip pre úlohy</Text>}
      <FlatList
        data={tasks}
        renderItem={({ item }) => (
          <TaskRow task={item} />
        )}
      />
    </View>
  );
}
```

FLATLIST · BEST PRACTICE

Udrž callback referenciu

Stabilizuj len potrebné props

- Použi `useCallback`, ak je funkcia použitá ako prop listu alebo jeho itemu
- `memo` pomáha preskočiť render ak sa jeho props nezmenili.

Stabilná referencia nepomôže, ak sa zároveň menia `data` alebo props každej položky.

SECTIONLIST

Keď položky patria do skupín

FlatList so sekciami

`SectionList` je virtualizovaný zoznam pre dáta rozdelené do skupín.

- `sections` obsahuje jednotlivé sekcie.
- Každá sekcia má typicky `title` a pole `data`.
- `renderItem` vykreslí položku v sekcii.

Použi ho napríklad pre kontakty podľa začiatočného písmena alebo správy podľa dňa.

```
const sections = [
  {
    title: 'A',
    data: [
      { id: '1', name: 'Adam' },
      { id: '2', name: 'Anna' },
    ],
  },
  {
    title: 'B',
    data: [{ id: '3', name: 'Bea' }],
  },
];
```

SECTIONLIST

Vykresli sekcie a ich položky

Dve úrovne vykresľovania

- `renderSectionHeader` vykreslí nadpis skupiny.
- `renderItem` vykreslí každú položku z `data`.
- `keyExtractor` vráti stabilný kľúč položky.

SectionHeader aj SectionItems zostávajú v jednom scrollovateľnom zozname.

```
<SectionList
  sections={sections}
  keyExtractor={(item) => item.id}
  renderSectionHeader={({ section }) => (
    <Text>{section.title}</Text>
  )}
  renderItem={({ item }) => (
    <Text>{item.name}</Text>
  )}
/>
```

SECTIONLIST

Sticky nadpis a separators

`stickySectionHeadersEnabled` ponechá nadpis aktuálnej sekcie pri hornom okraji počas scrollovania.

Oddelenie sekcií

`SectionSeparatorComponent` vykreslí obsah medzi sekciami, nie medzi každými dvoma položkami.

Na oddeľovanie jednotlivých položiek použi `ItemSeparatorComponent`.

```
<SectionList
  sections={sections}
  stickySectionHeadersEnabled
  renderItem={({ item }) => (
    <ContactRow contact={item} />
  )}
  renderSectionHeader={({ section }) => (
    <Text>{section.title}</Text>
  )}
  SectionSeparatorComponent={() => (
    <View style={{ height: 12 }} />
  )}
/>
```

SCROLLVIEW VS FLATLIST VS FLASHLIST

Ukážkové meranie · 5 000 položiek

Komponent	Načítanie	FPS	Pamäť
ScrollView	1–3 s	< 30	Vysoká
FlatList	~100 ms	~45	Stredná
FlashList	~50 ms	~54	Nízka

Orientačné výsledky jedného testu, nie univerzálny benchmark.

SCROLLVIEW VS FLATLIST VS FLASHLIST

ScrollView vs FlatList vs FlashList

Scenár	Odporúčanie
< 20 statických položiek	ScrollView
20–100 položiek	FlatList
> 100 položiek	FlashList
Rôzne typy položiek	FlashList + `getItemType`
Pevná výška položiek	`getItemLayout` / odhad rozmerov

Počty sú orientačné. Rozhodni podľa profilovania skutočného zoznamu.

OPTIMALIZÁCIA OBRÁZKOV

Každý frame má obmedzený čas

Rozpočet pri 60 FPS

Jeden frame trvá približne **16,7 ms**.

Do toho sa musí zmestiť JavaScript, layout (UI thread), natívne vykreslenie aj práca s obrázkami

Čo zaťažuje obrázky?

- Veľký súbor treba dekódovať.
- Obrázok môže byť väčší než jeho miesto v UI.
- Pri scrollovaní sa môžu opakovať načítania z cache.

OBRÁZKOVÝ PIPELINE

Čo prinášajú natívne image loadery?

Asynchrónne spracovanie

Načítanie a dekódovanie obrázkov mimo UI threadu.

Cache

Memory + disk cache môžu obmedziť opakované sťahovanie a dekódovanie.

Riadenie načítania

Priorita a prednačítanie pomáhajú pripraviť obrázky.

Príklady natívnych základov: **Glide** a **Coil** na Androide, **Nuke** a **SDWebImage** na iOS.

VÝBER KNIŽNICE

Expo Image alebo FastImage?

`expo-image`

Prirodzená voľba v Expo projekte.

- Placeholdery, napríklad BlurHash alebo ThumbHash
- Prechody pri zmene zdroja
- Priorita a prednačítanie

`react-native-fast-image`

Alternatíva používaná najmä v Bare React Native projektoch.

- Natívne cache správanie
- Preload API a priorita požiadaviek

ĎALŠIE KROKY

Zmeraj a dolaď obrázky v liste

Formát a rozmer

Zvoľ vhodný formát, napríklad WebP

Načítaj zdroj primeraný rozmeru, nie zbytočne veľký originál.

Priorita a preload

Uprednostni obrázky vo viewporte.

Prednačítaj blízke položky, ak to podporuje použitá knižnica.

Profilovanie

Sleduj JS aj UI thread počas scrollovania a renderovania.